Task 9 to 15- Complete check list and instructions

Task	Heading	Deliverable
9	Develop Student & Course Management APIs	Student and Course CRUD
10	JWT Authentication & Protected Routes	Secure login and protected endpoints
11	Database Schema & Tables/Collections	MongoDB models and ER diagram
12	Database Integration & CRUD Operations	Database-driven application
13	Real-Time Notification/Chat Using WebSockets	Live notifications and messaging
14	Optimization, Security & Error Handling	Secure and reliable backend
15	Deploy Application & Perform Testing	Live full-stack application

Task 9 — Develop Student & Course Management APIs

This task requires registration/login, student CRUD, course CRUD, validation, JSON responses, HTTP status codes, and Postman testing.

Activity 9.1 — Install required packages

UPDATE: Open a terminal inside your backend folder.

npm install express mongoose bcryptjs jsonwebtoken dotenv cors

We'll also use these packages in later tasks.

Activity 9.2 — Create the Student model

NEW, unless you already have a Mongoose User model:

backend/models/User.js

```
const mongoose = require("mongoose");


const userSchema = new mongoose.Schema(
 {
```

```
    name: {
      type: String,
      required: true,
      trim: true
    },
    email: {
      type: String,
      required: true,
      unique: true,
      lowercase: true,
      trim: true
    },
    password: {
      type: String,
      required: true,
      select: false
    },
    role: {
      type: String,
      enum: ["student", "admin"],
      default: "student"
    }
  },
  { timestamps: true }
);

module.exports = mongoose.model("User", userSchema);
```

If you already have a User or Student model, update the existing schema rather than registering a second model for the same users.

Activity 9.3 — Create the Course model

NEW, only if you do not already have a Course model:

backend/models/Course.js

```
const mongoose = require("mongoose");


const courseSchema = new mongoose.Schema(
 {
  title: {
   type: String,
   required: true,
   trim: true
  },
  description: {
   type: String,
   required: true,
   trim: true
  },
  duration: {
   type: Number,
   required: true,
   min: 1
  }
 },
 { timestamps: true }
);
```

```
module.exports = mongoose.model("Course", courseSchema);
```

This example measures duration as a positive number; document whether your project uses hours or weeks. Preserve any additional fields already used by your existing Course API.

Activity 9.4 — Student registration and login

UPDATE: backend/controllers/authController.js

Your registration must hash passwords and create only student accounts. A secure implementation of the registration function is:

```
const bcrypt = require("bcryptjs");

const User = require("../models/User");


async function register(req, res, next) {
 try {
  const { name, email, password } = req.body;


  if (
   typeof name !== "string" ||
   !name.trim() ||
   typeof email !== "string" ||
   !/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email) ||
   typeof password !== "string" ||
   password.length < 12
  ) {
   return res.status(400).json({
    message: "Valid name, email and password of at least 12 characters are required"
   });
  }


  const normalizedEmail = email.trim().toLowerCase();
```

```
const existing = await User.findOne({
 email: normalizedEmail
});

if (existing) {
 return res.status(409).json({
  message: "Email already registered"
 });
}

const hashedPassword = await bcrypt.hash(password, 12);

const user = await User.create({
 name: name.trim(),
 email: normalizedEmail,
 password: hashedPassword,
 role: "student"
});

return res.status(201).json({
 message: "Registration successful",
 student: {
  id: user._id,
  name: user.name,
  email: user.email,
  role: user.role
 }
```

```
  });
 } catch (error) {
  next(error);
 }
}
```

Do not replace your entire authentication controller with this function. Keep your working login and existing exports; if your current registration already performs equivalent validation and hashing, retain it.

Activity 9.5 — Student CRUD controller

NEW: backend/controllers/studentController.js

```
const mongoose = require("mongoose");
const bcrypt = require("bcryptjs");
const User = require("../models/User");

const publicFields = "name email role createdAt updatedAt";

async function getStudents(req, res, next) {
 try {
  const students = await User.find({
   role: "student"
  }).select(publicFields);

  res.json({ students });
 } catch (error) {
  next(error);
 }
}

async function getStudentById(req, res, next) {
```

```
  try {
    if (!mongoose.isValidObjectId(req.params.id)) {
      return res.status(400).json({
        message: "Invalid student ID"
      });
    }

    const student = await User.findOne({
      _id: req.params.id,
      role: "student"
    }).select(publicFields);

    if (!student) {
      return res.status(404).json({
        message: "Student not found"
      });
    }

    res.json({ student });
  } catch (error) {
    next(error);
  }
}

async function createStudent(req, res, next) {
  try {
    const { name, email, password } = req.body;
```

```
if (
  typeof name !== "string" ||
  !name.trim() ||
  typeof email !== "string" ||
  !/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email) ||
  typeof password !== "string" ||
  password.length < 12
) {
  return res.status(400).json({
    message: "Valid name, email and password are required"
  });
}

const normalizedEmail = email.trim().toLowerCase();

if (await User.exists({ email: normalizedEmail })) {
  return res.status(409).json({
    message: "Email already registered"
  });
}

const student = await User.create({
  name: name.trim(),
  email: normalizedEmail,
  password: await bcrypt.hash(password, 12),
  role: "student"
});
```

```
    res.status(201).json({
      message: "Student created",
      student: {
        id: student.id,
        name: student.name,
        email: student.email,
        role: student.role
      }
    });
  } catch (error) {
    next(error);
  }
}

async function updateStudent(req, res, next) {
  try {
    const { id } = req.params;

    if (!mongoose.isValidObjectId(id)) {
      return res.status(400).json({
        message: "Invalid student ID"
      });
    }

    const updates = {};

    if (req.body.name !== undefined) {
      if (
```

```
    typeof req.body.name !== "string" ||
    !req.body.name.trim()
  ) {
    return res.status(400).json({
      message: "Invalid name"
    });
  }

  updates.name = req.body.name.trim();
}

if (req.body.email !== undefined) {
  if (
    typeof req.body.email !== "string" ||
    !/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(req.body.email)
  ) {
    return res.status(400).json({
      message: "Invalid email"
    });
  }

  updates.email = req.body.email.trim().toLowerCase();
}

if (Object.keys(updates).length === 0) {
  return res.status(400).json({
    message: "No valid fields supplied"
  });
```

```
    }

    const student = await User.findOneAndUpdate(
      { _id: id, role: "student" },
      { $set: updates },
      { new: true, runValidators: true }
    ).select(publicFields);

    if (!student) {
      return res.status(404).json({
        message: "Student not found"
      });
    }

    res.json({
      message: "Student updated",
      student
    });
  } catch (error) {
    next(error);
  }
}

async function deleteStudent(req, res, next) {
  try {
    if (!mongoose.isValidObjectId(req.params.id)) {
      return res.status(400).json({
        message: "Invalid student ID"
```

```
      });
    }

    const student = await User.findOneAndDelete({
      _id: req.params.id,
      role: "student"
    });

    if (!student) {
      return res.status(404).json({
        message: "Student not found"
      });
    }

    res.json({ message: "Student deleted" });
  } catch (error) {
    next(error);
  }
}

module.exports = {
  getStudents,
  getStudentById,
  createStudent,
  updateStudent,
  deleteStudent
};
```

These are admin-managed student endpoints. Student self-service profile endpoints can be added separately.

Deleting a student here deletes only the user document. In Task 12, we'll address dependent enrollments, progress records, and messages so deletion doesn't leave inconsistent data.

Activity 9.6 — Student routes

NEW: backend/routes/studentRoutes.js

```
const express = require("express");

const router = express.Router();


const authMiddleware = require("../middleware/authMiddleware");

const authorizeRoles = require("../middleware/roleMiddleware");


const controller = require("../controllers/studentController");


router.use(authMiddleware);

router.use(authorizeRoles("admin"));


router.get("/", controller.getStudents);

router.get("/:id", controller.getStudentById);

router.post("/", controller.createStudent);

router.put("/:id", controller.updateStudent);

router.delete("/:id", controller.deleteStudent);


module.exports = router;
```

UPDATE: backend/server.js

Add:

```
const studentRoutes = require("./routes/studentRoutes");
```

app.use("/api/v1/students", studentRoutes);

Place the app.use line after Express app initialization and before your centralized error handler.

Activity 9.7 — Course CRUD

You already implemented Course CRUD in our earlier work, so do not replace it unnecessarily.

Verifrouter.get("/", courseController.getAllCourses);

router.get("/:id", courseController.getCourseById);

router.post(
 "/",
 authMiddleware,
 authorizeRoles("admin"),
 courseController.createCourse
);

router.put(
 "/:id",
 authMiddleware,
 authorizeRoles("admin"),
 courseController.updateCourse
);

router.delete(
 "/:id",
 authMiddleware,
 authorizeRoles("admin"),
 courseController.deleteCourse
);

Your controller function names may differ. Retain the working names and verify that controllers validate required fields, use Mongoose operations, and return appropriate HTTP status codes.

Activity 9.8 — Postman testing

Method	Endpoint	Expected
POST	/api/v1/auth/register	Student registered
POST	/api/v1/auth/login	Login successful
GET	/api/v1/students	Student list, admin only
GET	/api/v1/students/:id	Student details
POST	/api/v1/students	Student created
PUT	/api/v1/students/:id	Student updated
DELETE	/api/v1/students/:id	Student deleted
GET	/api/v1/courses	Course list
POST	/api/v1/courses	Course created by admin
PUT	/api/v1/courses/:id	Course updated by admin
DELETE	/api/v1/courses/:id	Course deleted by admin

Test valid requests, missing fields, invalid IDs, duplicate emails, and unauthorized access. The PDF expects successful Postman testing, not just working route definitions.

Task 9 deliverable: Working Student and Course REST APIs with validation and correct responses.

Task 10 — JWT Authentication & Protected Routes

Activity 10.1 — Configure JWT

UPDATE: backend/.env

JWT_SECRET=replace_with_a_long_random_secret

JWT_EXPIRES_IN=1h

Generate a secret using:

node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"

Paste the generated value into .env. Never commit or share it.

Activity 10.2 — Login and JWT generation

UPDATE: backend/controllers/authController.js

Keep the registration function from Task 9. Use the following login implementation, ensuring imports are declared only once.

```
const jwt = require("jsonwebtoken");

const bcrypt = require("bcryptjs");

const User = require("../models/User");


async function login(req, res, next) {
 try {
  const { email, password } = req.body;


  if (
   typeof email !== "string" ||
   typeof password !== "string"
  ) {
   return res.status(400).json({
    message: "Email and password are required"
   });
  }
```

```
const user = await User.findOne({
  email: email.trim().toLowerCase()
}).select("+password");

if (!user) {
  return res.status(401).json({
    message: "Invalid credentials"
  });
}

const valid = await bcrypt.compare(
  password,
  user.password
);

if (!valid) {
  return res.status(401).json({
    message: "Invalid credentials"
  });
}

if (!process.env.JWT_SECRET) {
  throw new Error("JWT_SECRET is missing");
}

const token = jwt.sign(
  {
```

```
      email: user.email,
      role: user.role
    },
    process.env.JWT_SECRET,
    {
      subject: String(user._id),
      expiresIn: process.env.JWT_EXPIRES_IN || "1h"
    }
  );

  return res.json({
    message: "Login successful",
    token,
    tokenType: "Bearer",
    expiresIn: process.env.JWT_EXPIRES_IN || "1h",
    student: {
      id: user._id,
      name: user.name,
      email: user.email,
      role: user.role
    }
  });
 } catch (error) {
  next(error);
 }
}
```

UPDATE: Keep one export statement at the bottom:

```
module.exports = { register, login };
```

This assumes registration is named register. Use your actual registration function name if different.

Activity 10.3 — Authentication middleware

UPDATE or NEW: backend/middleware/authMiddleware.js

```
const jwt = require("jsonwebtoken");


function authMiddleware(req, res, next) {
 const header = req.headers.authorization;


 if (!header || !header.startsWith("Bearer ")) {
  return res.status(401).json({
   message: "Authentication required"
  });
 }


 const token = header.slice(7);


 try {
  const decoded = jwt.verify(
   token,
   process.env.JWT_SECRET,
   { algorithms: ["HS256"] }
  );


  req.user = decoded;
  next();
 } catch (error) {
  return res.status(401).json({
```

```
    message: "Invalid or expired token"
   });
 }
}
module.exports = authMiddleware;
```

For higher-security deployments, validate the user's current database status and role as well, rather than relying indefinitely on roles embedded in an earlier token.

Activity 10.4 — Role authorization middleware

UPDATE or NEW: backend/middleware/roleMiddleware.js

```
function authorizeRoles(...allowedRoles) {
 return (req, res, next) => {
  if (!req.user) {
   return res.status(401).json({
    message: "Authentication required"
   });
  }

  if (!allowedRoles.includes(req.user.role)) {
   return res.status(403).json({
    message: "Access denied"
   });
  }
  next();
 };
}
module.exports = authorizeRoles;
```

Activity 10.5 — Protect the routes

Apply authentication before authorization.

```
router.get("/", courseController.getAllCourses);


router.post(
 "/",
 authMiddleware,
 authorizeRoles("admin"),
 courseController.createCourse
);
```

Use the same protection for course update and delete. Student profile and dashboard endpoints should require authentication, with ownership or role checks appropriate to the endpoint.

Activity 10.6 — Store and send JWT in React

UPDATE: frontend/src/api/apiClient.js

If you already have an Axios instance, add only the interceptor.

```
import axios from "axios";


const apiClient = axios.create({
 baseURL:
  import.meta.env.VITE_API_URL ||
  "http://localhost:5000/api/v1"
});


apiClient.interceptors.request.use((config) => {
 const token = sessionStorage.getItem("token");


 if (token) {
  config.headers.Authorization = `Bearer ${token}`;
 }
 return config;
```

```
});
export default apiClient;
```

UPDATE: Your existing login component, after a successful login:

```
sessionStorage.setItem("token", response.data.token);
sessionStorage.setItem(
  "role",
  response.data.student.role
);
```

On logout:

```
sessionStorage.removeItem("token");
sessionStorage.removeItem("role");
```

Then update your React authentication state and navigate to the login page.

For this educational implementation, sessionStorage is used for simplicity. It is accessible to JavaScript and therefore vulnerable if an XSS flaw exists; a production authentication design should assess secure HttpOnly cookies, CSRF protection, and session revocation.

Activity 10.7 — Authentication tests

Verify the complete workflow:

1. Register a student.
2. Log in and receive a JWT.
3. Access an authenticated endpoint with the token.
4. Repeat without the token and expect 401.
5. Repeat with an invalid or expired token and expect 401.
6. Verify a student cannot access admin-only endpoints (403).
7. Log out and confirm the frontend clears the token and authentication state.

Task 10 deliverable: Working JWT authentication and protected API routes.

Task 11 — Database Schema & Tables/Collections

This task is about the database design, including entities, relationships, identifiers, constraints, and documentation.

You already have User, Course, Enrollment, and LearningProgress models from our earlier activities.

Activity 11.1 — Confirm MongoDB configuration

UPDATE: backend/.env

MONGO_URI=mongodb://127.0.0.1:27017/faceprep_course_system

This is an example for local MongoDB. If you use MongoDB Atlas, keep your existing Atlas connection string instead.

Activity 11.2 — User and Course collections

Use the User.js and Course.js schemas from Task 9.

MongoDB automatically generates _id values for these documents.

Activity 11.3 — Enrollment collection

UPDATE or NEW: backend/models/Enrollment.js

```
const mongoose = require("mongoose");


const enrollmentSchema = new mongoose.Schema(
 {
   student: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User",
    required: true
   },
   course: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "Course",
```

```
    required: true
   },
   status: {
    type: String,
    enum: ["active", "completed", "withdrawn"],
    default: "active"
   },
   enrolledAt: {
    type: Date,
    default: Date.now
   }
  },
  { timestamps: true }
);

enrollmentSchema.index(
  { student: 1, course: 1 },
  { unique: true }
);

module.exports = mongoose.model(
  "Enrollment",
  enrollmentSchema
);
```

This adds withdrawn so we can preserve enrollment history rather than deleting it permanently.

Important integration change: Your earlier unenrollment controller used findOneAndDelete(). If you adopt withdrawn, replace the deletion with a status update,

and update enrollment logic to reactivate withdrawn records instead of inserting duplicates. Otherwise, the unique index will prevent re-enrollment.

Activity 11.4 — Learning Progress collection

Keep your existing:

backend/models/LearningProgress.js

It should contain student and course references, completed lessons, total lessons, progress percentage, and timestamps.

Maintain the unique compound index on { student, course }.

Activity 11.5 — Notification collection

NEW: backend/models/Notification.js

```
const mongoose = require("mongoose");


const notificationSchema = new mongoose.Schema(
 {
  recipient: {
   type: mongoose.Schema.Types.ObjectId,
   ref: "User",
   required: true
  },
  message: {
   type: String,
   required: true,
   trim: true,
   maxlength: 500
  },
  isRead: {
   type: Boolean,
   default: false
  }
```

```
  },
  { timestamps: true }
);

notificationSchema.index({
  recipient: 1,
  createdAt: -1
});

module.exports = mongoose.model(
  "Notification",
  notificationSchema
);
```

Activity 11.6 — Message collection

NEW: backend/models/Message.js

```
const mongoose = require("mongoose");

const messageSchema = new mongoose.Schema(
 {
   sender: {
     type: mongoose.Schema.Types.ObjectId,
     ref: "User",
     required: true
   },
   recipient: {
     type: mongoose.Schema.Types.ObjectId,
     ref: "User",
     required: true
```

```
    },
    text: {
      type: String,
      required: true,
      trim: true,
      maxlength: 2000
    }
  },
  { timestamps: true }
);

messageSchema.index({
  sender: 1,
  recipient: 1,
  createdAt: -1
});

module.exports = mongoose.model(
  "Message",
  messageSchema
);
```

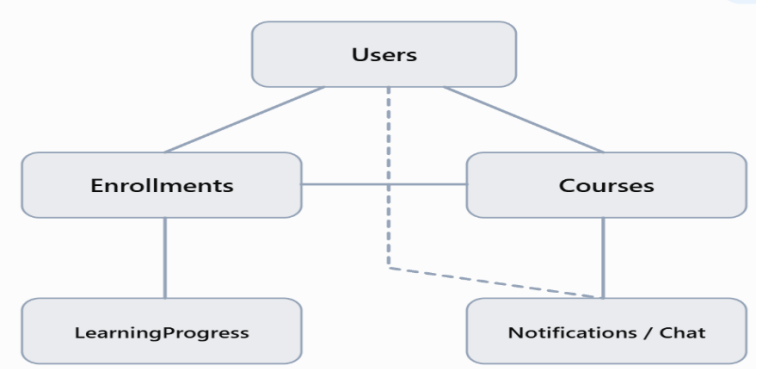
Activity 11.7 — Document database relationships

Create:

docs/database-schema.md

Include the following diagram:

DATABASE RELATIONSHIPS



Conceptual relationship diagram. Notifications and chat messages reference users; enrollment and progress reference both users and courses.

Also document each collection's fields, required constraints, and unique indexes.

Task 11 deliverable: Working MongoDB schemas plus documented relationships/ER diagram.

Task 12 — Database Integration & CRUD Operations

This task requires a fully database-driven application, including registration, login, student profile CRUD, course CRUD, database errors, and frontend/Postman testing.

Activity 12.1 — Configure MongoDB connection

NEW, if it doesn't exist: backend/config/db.js

```
const mongoose = require("mongoose");

async function connectDB() {
 if (!process.env.MONGO_URI) {
  throw new Error("MONGO_URI is not configured");
 }
 await mongoose.connect(process.env.MONGO_URI);
 console.log("MongoDB connected successfully");
}

module.exports = connectDB;
```

Activity 12.2 — Connect the database before starting Express

UPDATE: backend/server.js

Your startup should follow this pattern:

```
require("dotenv").config();

const express = require("express");
const connectDB = require("./config/db");

const app = express();

app.use(express.json());

// Keep your existing CORS, routes, and error middleware here.

const PORT = process.env.PORT || 5000;

async function startServer() {
 try {
  await connectDB();

  app.listen(PORT, () => {
   console.log(` Server running on port ${PORT}` );
  });
 } catch (error) {
  console.error(
   "Unable to start server:",
   error.message
  );
  process.exit(1);
```

```
 }
}


startServer();
```

Do not paste this as a second server implementation. Merge the database connection and startup logic into your existing server.js, preserving all current route registrations.

Activity 12.3 — Connect registration and login to MongoDB

The registration and login implementations in Tasks 9 and 10 already use:

```
await User.create(...);

await User.findOne(...);
```

Therefore, no additional temporary student array is needed.

UPDATE: Remove the old in-memory student storage from your authentication service only after confirming both registration and login use the same Mongoose User model.

Do not maintain two separate sources of student accounts.

Activity 12.4 — Create a student profile API

The PDF expects student profile retrieval, update, and deletion.

NEW: backend/controllers/profileController.js

```
const User = require("../models/User");


async function getProfile(req, res, next) {
 try {
  const user = await User.findById(
   req.user.sub
  ).select("name email role createdAt");


  if (!user) {
   return res.status(404).json({
    message: "Profile not found"
   });
```

```
    }

    res.json({ user });
  } catch (error) {
    next(error);
  }
}

async function updateProfile(req, res, next) {
  try {
    const { name } = req.body;

    if (
      typeof name !== "string" ||
      !name.trim()
    ) {
      return res.status(400).json({
        message: "Valid name is required"
      });
    }

    const user = await User.findByIdAndUpdate(
      req.user.sub,
      { $set: { name: name.trim() } },
      { new: true, runValidators: true }
    ).select("name email role");

    if (!user) {
```

```
    return res.status(404).json({
      message: "Profile not found"
    });
   }
   res.json({
    message: "Profile updated",
    user
   });
 } catch (error) {
   next(error);
 }
}
module.exports = {
 getProfile,
 updateProfile
};
```

NEW: backend/routes/profileRoutes.js

```
const express = require("express");
const router = express.Router();
const authMiddleware = require("../middleware/authMiddleware");
const {
 getProfile,
 updateProfile
} = require("../controllers/profileController");

router.get("/", authMiddleware, getProfile);
```

```
router.put("/", authMiddleware, updateProfile);

module.exports = router;
```

UPDATE: backend/server.js

```
const profileRoutes = require("./routes/profileRoutes");

app.use("/api/v1/profile", profileRoutes);
```

Student deletion is available through the admin CRUD API from Task 9. For a production-quality deletion workflow, remove or anonymize dependent records in a controlled operation, preferably using a MongoDB transaction where supported. Do not expose unrestricted profile deletion until that cleanup is implemented.

Activity 12.5 — Database error handling

NEW: backend/middleware/errorHandler.js

```
function errorHandler(err, req, res, next) {
 console.error(err);
 if (err.code === 11000) {
  return res.status(409).json({
   message: "Duplicate value already exists"
  });
 }
 if (err.name === "ValidationError") {
  return res.status(400).json({
   message: "Validation failed",
   errors: Object.values(err.errors).map(
    (item) => item.message
   )
  });
 }
 if (err.name === "CastError") {
  return res.status(400).json({
```

```
    message: "Invalid identifier"
  });
 }
 const status =
  Number.isInteger(err.status) &&
  err.status >= 400 &&
  err.status < 600
   ? err.status
   : 500;
 res.status(status).json({
  message:
   status === 500
    ? "Internal server error"
    : err.message
 });
}
module.exports = errorHandler;
```

UPDATE: backend/server.js

After all API routes:

```
const errorHandler = require("./middleware/errorHandler");


app.use(errorHandler);
```

This must be the last error-handling middleware.

Activity 12.6 — Verify Course CRUD uses MongoDB

Check that your Course controller uses Mongoose operations such as:

```
Course.create(...)
Course.find(...)
```

Course.findById(...)

Course.findByIdAndUpdate(...)

Course.findByIdAndDelete(...)

For updates, use { new: true, runValidators: true } where appropriate.

Validate ObjectIds and required fields before database operations.

Activity 12.7 — Test through Postman and React

Test this complete workflow:

1. Register a student and verify the user document exists in MongoDB.
2. Restart the backend and log in again; the student should still exist.
3. Retrieve the student's profile.
4. Update the student's name and verify it persists.
5. Use the admin account to create a course.
6. Retrieve, update, and delete a test course.
7. Verify invalid IDs, duplicate emails, and missing required fields return appropriate errors.
8. Verify React displays the updated database records.

Task 12 deliverable: Fully database-driven Student and Course APIs.

Task 13 — Real-Time Notification/Chat Using WebSockets

We'll use Socket.IO, which provides real-time communication between React and Express.

Activity 13.1 — Install Socket.IO

In your backend folder:

npm install socket.io

In your frontend folder:

npm install socket.io-client

Activity 13.2 — Create the Socket.IO server

NEW: backend/config/socket.js

```
const { Server } = require("socket.io");
const jwt = require("jsonwebtoken");
const mongoose = require("mongoose");

const User = require("../models/User");
const Message = require("../models/Message");
const Notification = require("../models/Notification");

function initializeSocket(httpServer) {
 const io = new Server(httpServer, {
  cors: {
   origin: process.env.CLIENT_URL,
   methods: ["GET", "POST"]
  }
 });

 // Authenticate the socket connection
 io.use(async (socket, next) => {
  try {
   const token = socket.handshake.auth.token;

   if (!token) {
    return next(
     new Error("Authentication required")
    );
   }
```

```
    const decoded = jwt.verify(
      token,
      process.env.JWT_SECRET,
      { algorithms: ["HS256"] }
    );
    const user = await User.findById(decoded.sub)
      .select("_id name role");
    if (!user) {
      return next(
        new Error("User not found")
      );
    }
    socket.user = {
      id: String(user._id),
      name: user.name,
      role: user.role
    };
    next();
  } catch (error) {
    next(new Error("Invalid authentication token"));
  }
});
io.on("connection", (socket) => {
  const userId = socket.user.id;
  // Each authenticated user has a private room
  socket.join(`user:${userId}`);
  console.log("Socket connected:", userId);
  socket.on("send_message", async (data, ack) => {
```

```
try {
  const { recipientId, text } = data || {};
  if (
    !mongoose.isValidObjectId(recipientId) ||
    typeof text !== "string" ||
    !text.trim() ||
    text.trim().length > 2000
  ) {
    throw new Error("Invalid message data");
  }
  const recipient = await User.findById(
    recipientId
  ).select("_id");
  if (!recipient) {
    throw new Error("Recipient not found");
  }
  const message = await Message.create({
    sender: userId,
    recipient: recipientId,
    text: text.trim()
  });
  const payload = {
    _id: message._id,
    sender: userId,
    recipient: String(recipientId),
    text: message.text,
    createdAt: message.createdAt
  };
```

```
      io.to(`user:${userId}`)
        .to(`user:${recipientId}`)
        .emit("receive_message", payload);
      const notification = await Notification.create({
        recipient: recipientId,
        message: `New message from ${socket.user.name}`
      });
      io.to(`user:${recipientId}`).emit(
        "new_notification",
        notification
      );
      if (typeof ack === "function") {
        ack({ success: true });
      }
    } catch (error) {
      if (typeof ack === "function") {
        ack({
          success: false,
          message: error.message
        });
      }
    }
  });
  socket.on("disconnect", () => {
    console.log("Socket disconnected:", userId);
  });
});
return io;
```

}

module.exports = initializeSocket;

This implementation authenticates each socket, stores messages, creates notifications, and sends events to the intended users.

It is a basic direct-message implementation. Before production use, add recipient permissions, message rate limits, and stronger abuse controls. The client cannot choose the sender identity because it comes from the authenticated socket.

Activity 13.3 — Connect Socket.IO to Express

UPDATE: backend/server.js

At the top:

const http = require("http");

const initializeSocket = require("./config/socket");

After creating your Express app:

const server = http.createServer(app);

initializeSocket(server);

REPLACE: Your existing final app.listen(...) call with:

server.listen(PORT, () => {

  console.log(` Server running on port ${PORT}`);

});

Do this inside your existing startup function after MongoDB connects. There must be only one HTTP server listening on the port.

Activity 13.4 — Configure frontend origin

UPDATE: backend/.env

CLIENT_URL=http://localhost:5173

Replace the port if your React frontend runs elsewhere.

Activity 13.5 — Create the React socket client

NEW: frontend/src/api/socketClient.js

```
import { io } from "socket.io-client";

const SOCKET_URL =
  import.meta.env.VITE_SOCKET_URL ||
  "http://localhost:5000";

export function createSocket() {
  return io(SOCKET_URL, {
    autoConnect: false,
    auth: {
      token: sessionStorage.getItem("token")
    }
  });
}
```

Use the backend origin here, without /api/v1.

Activity 13.6 — Create a Chat component

NEW: frontend/src/components/Chat.jsx

```
import { useEffect, useState } from "react";
import { createSocket } from "../api/socketClient";

function Chat() {
  const [socket, setSocket] = useState(null);
  const [connected, setConnected] = useState(false);
  const [recipientId, setRecipientId] = useState("");
  const [message, setMessage] = useState("");
  const [messages, setMessages] = useState([]);
  const [error, setError] = useState("");
```

```
useEffect(() => {
 const client = createSocket();

 client.on("connect", () => {
  setConnected(true);
  setError("");
 });

 client.on("disconnect", () => {
  setConnected(false);
 });

 client.on("connect_error", (err) => {
  setError(err.message);
 });

 client.on("receive_message", (data) => {
  setMessages((previous) => [
   ...previous,
   data
  ]);
 });

 client.connect();
 setSocket(client);

 return () => {
  client.disconnect();
```

```
  };
}, []);

function sendMessage(event) {
  event.preventDefault();

  if (!socket || !connected) return;

  socket.emit(
    "send_message",
    {
      recipientId,
      text: message
    },
    (result) => {
      if (result.success) {
        setMessage("");
      } else {
        setError(result.message);
      }
    }
  );
}

return (
  <section>
    <h2>Student Chat</h2>
```

```
<p>
 Connection:{" "}
 {connected ? "Online" : "Offline"}
</p>

{error && <p role="alert">{error}</p>}

<form onSubmit={sendMessage}>
 <input
  value={recipientId}
  onChange={(e) =>
   setRecipientId(e.target.value)
  }
  placeholder="Recipient MongoDB user ID"
  required
 />

 <input
  value={message}
  onChange={(e) =>
   setMessage(e.target.value)
  }
  placeholder="Type a message"
  maxLength={2000}
  required
 />

 <button
```

```
        type="submit"
        disabled={!connected}
       >
        Send
       </button>
     </form>

     <h3>Messages</h3>

     {messages.map((item) => (
       <p key={item._id}>
         <strong>{item.sender}:</strong>{" "}
         {item.text}
       </p>
     ))}
   </section>
 );
}

export default Chat;
```

This is the basic real-time chat interface. It receives messages without a page refresh. It uses MongoDB user IDs for testing; a user-selection interface would improve usability.

Activity 13.7 — Add Chat to React

UPDATE: frontend/src/App.jsx

Add the import:

import Chat from "./components/Chat";

Render it inside your authenticated section:

```
{sessionStorage.getItem("token") && (
  <Chat />
)}
```

If you're using React Router, put it inside an authenticated route instead.

Activity 13.8 — Create message history API

Real-time events alone won't restore messages after refreshing the page.

NEW: backend/routes/messageRoutes.js

```
const express = require("express");
const mongoose = require("mongoose");
const Message = require("../models/Message");

const authMiddleware = require("../middleware/authMiddleware");

const router = express.Router();

router.get("/:userId", authMiddleware, async (req, res, next) => {
  try {
    const myId = req.user.sub;
    const otherId = req.params.userId;

    if (
      !mongoose.isValidObjectId(myId) ||
      !mongoose.isValidObjectId(otherId)
    ) {
      return res.status(400).json({
        message: "Invalid user ID"
      });
    }
```

```
    const messages = await Message.find({
      $or: [
        { sender: myId, recipient: otherId },
        { sender: otherId, recipient: myId }
      ]
    })
      .sort({ createdAt: -1 })
      .limit(100)
      .lean();

    res.json({ messages: messages.reverse() });
  } catch (error) {
    next(error);
  }
});
module.exports = router;
```

UPDATE: backend/server.js

```
const messageRoutes = require("./routes/messageRoutes");

app.use("/api/v1/messages", messageRoutes);
```

UPDATE: In Chat.jsx, import your Axios client:

```
import apiClient from "../api/apiClient";
```

Add this function inside the component:

```
async function loadHistory() {
  if (!recipientId) return;
```

```
 try {
   const response = await apiClient.get(
     `/messages/${recipientId}`
   );

   setMessages(response.data.messages);
 } catch (error) {
   setError("Unable to load message history");
 }
}
```

Add a button next to the recipient field:

```
<button type="button" onClick={loadHistory}>
  Load Chat History
</button>
```

For a polished chat, filter incoming events by the selected conversation and reset messages when switching recipients.

Activity 13.9 — Create notification API

NEW: backend/routes/notificationRoutes.js

```
const express = require("express");
const Notification = require("../models/Notification");
const authMiddleware = require("../middleware/authMiddleware");

const router = express.Router();

router.get("/", authMiddleware, async (req, res, next) => {
  try {
    const notifications = await Notification.find({
      recipient: req.user.sub
```

```
    })
      .sort({ createdAt: -1 })
      .limit(50);

    res.json({ notifications });
  } catch (error) {
    next(error);
  }
});

router.patch("/:id/read", authMiddleware, async (req, res, next) => {
  try {
    const notification =
      await Notification.findOneAndUpdate(
        {
          _id: req.params.id,
          recipient: req.user.sub
        },
        { $set: { isRead: true } },
        { new: true }
      );

    if (!notification) {
      return res.status(404).json({
        message: "Notification not found"
      });
    }
```

```
    res.json({ notification });
  } catch (error) {
    next(error);
  }
});
module.exports = router;
```

UPDATE: backend/server.js

```
const notificationRoutes = require("./routes/notificationRoutes");

app.use("/api/v1/notifications", notificationRoutes);
```

Activity 13.10 — Display notifications in React

NEW: frontend/src/components/Notifications.jsx

```
import { useEffect, useState } from "react";
import apiClient from "../api/apiClient";
import { createSocket } from "../api/socketClient";

function Notifications() {
  const [notifications, setNotifications] = useState([]);

  useEffect(() => {
    let active = true;

    apiClient.get("/notifications").then((response) => {
      if (active) {
        setNotifications(response.data.notifications);
      }
    }).catch(console.error);
```

```
  const socket = createSocket();

  socket.on("new_notification", (notification) => {
    setNotifications((previous) => [
      notification,
      ...previous
    ]);
  });

  socket.connect();

  return () => {
    active = false;
    socket.disconnect();
  };
}, []);

async function markRead(id) {
  await apiClient.patch(
    `/notifications/${id}/read`
  );

  setNotifications((previous) =>
    previous.map((item) =>
      item._id === id
        ? { ...item, isRead: true }
        : item
```

```
    )
  );
 }

 return (
  <section>
   <h2>Notifications</h2>

   {notifications.length === 0 && (
    <p>No notifications yet.</p>
   )}

   {notifications.map((item) => (
    <div key={item._id}>
     <p>{item.message}</p>

     {!item.isRead && (
      <button
       onClick={() => markRead(item._id)}
      >
       Mark as Read
      </button>
     )}
    </div>
   ))}
  </section>
 );
}
```

export default Notifications;

UPDATE: frontend/src/App.jsx

Import and render the component inside the authenticated area:

import Notifications from "./components/Notifications";

{sessionStorage.getItem("token") && (

<Notifications />

)}

Activity 13.11 — Test real-time communication

Open two browser windows, preferably separate browser profiles.

1. Log in as Student A in the first window.
2. Log in as Student B in the second.
3. Open the Chat page in both windows.
4. Enter the other student's MongoDB user ID.
5. Send a message from Student A.
6. Verify Student B receives it without refreshing.
7. Check that Student B receives a notification.
8. Verify connection status changes when the socket disconnects.
9. Refresh and load chat history.
10. Verify unauthenticated socket connections are rejected.

Task 13 deliverable: Real-time messaging and notifications, with persistent messages and connection status.

Task 14 — Optimization, Security & Error Handling

Activity 14.1 — Configure CORS UPDATE: backend/server.js

Install CORS if needed:

```
npm install cors
```

Add:

```
const cors = require("cors");
const allowedOrigins = (
  process.env.CLIENT_URL || "http://localhost:5173"
)
  .split(",")
  .map((origin) => origin.trim());
app.use(
  cors({
    origin(origin, callback) {
      if (!origin || allowedOrigins.includes(origin)) {
        return callback(null, true);
      }
      callback(
        Object.assign(
          new Error("Origin not allowed by CORS"),
          { status: 403 }
        )
      );
    }
  })
);
```

Put this before your API routes. Remove duplicate CORS middleware if you already have it.

For Socket.IO, use the same trusted origin configuration. Don't use unrestricted origins in production.

Activity 14.2 — Add request validation

Install:

npm install express-validator

NEW: backend/middleware/validateRequest.js

```
const { validationResult } = require("express-validator");

function validateRequest(req, res, next) {
  const errors = validationResult(req);

  if (!errors.isEmpty()) {
    return res.status(400).json({
      message: "Validation failed",
      errors: errors.array().map((error) => ({
        field: error.path,
        message: error.msg
      }))
    });
  }

  next();
}
module.exports = validateRequest;
```

NEW: backend/validators/authValidators.js

```
const { body } = require("express-validator");

const registerValidation = [
```

```
  body("name")
    .isString()
    .trim()
    .notEmpty()
    .withMessage("Name is required"),

  body("email")
    .isEmail()
    .withMessage("Valid email is required")
    .normalizeEmail(),

  body("password")
    .isString()
    .isLength({ min: 12 })
    .withMessage(
      "Password must contain at least 12 characters"
    )
];

const loginValidation = [
  body("email").isEmail(),
  body("password").isString().notEmpty()
];

module.exports = {
  registerValidation,
  loginValidation
};
```

UPDATE: Your existing authentication routes.

Add imports:

```
const {
  registerValidation,
  loginValidation
} = require("../validators/authValidators");


const validateRequest = require("../middleware/validateRequest");
```

Then attach validation before the controllers:

```
router.post(
  "/register",
  registerValidation,
  validateRequest,
  authController.register
);


router.post(
  "/login",
  loginValidation,
  validateRequest,
  authController.login
);
```

Keep your actual controller function names.

For course and student CRUD, add equivalent field validation, including length limits and ObjectId checks. Continue validating in controllers and Mongoose schemas too; client-side validation alone is insufficient.

Activity 14.3 — Verify password hashing

Your registration code should hash passwords:

```
const hashedPassword = await bcrypt.hash(
  password,
  12
);
```

Your login code should compare them:

```
const valid = await bcrypt.compare(
  password,
  user.password
);
```

Verify in MongoDB that the stored password is a hash, not plaintext.

If your model already hashes passwords in a Mongoose save hook, don't hash them again in the controller.

Activity 14.4 — Centralized error handling

Use backend/middleware/errorHandler.js from Task 12.

UPDATE: backend/server.js

After all API routes, add a 404 handler:

```
app.use((req, res) => {
  res.status(404).json({
    message: "API endpoint not found"
  });
});
```

Then register the error handler:

```
app.use(errorHandler);
```

The order must be:

1. CORS and request parsing.
2. API routes.
3. Unknown-route handler.

4. Centralized error handler.

Activity 14.5 — Add basic security headers

Install:

npm install helmet

UPDATE: backend/server.js

```
const helmet = require("helmet");


app.use(helmet());
```

Add it before your routes.

Review its Content Security Policy if your application later serves HTML directly from Express.

Activity 14.6 — Add API rate limiting

Install:

npm install express-rate-limit

NEW: backend/middleware/loginLimiter.js

```
const { rateLimit } = require("express-rate-limit");


const loginLimiter = rateLimit({
 windowMs: 15 * 60 * 1000,
 limit: 10,
 standardHeaders: "draft-8",
 legacyHeaders: false,
 message: {
  message: "Too many login attempts. Try again later."
 }
});


module.exports = loginLimiter;
```

UPDATE: Your authentication routes.

```
const loginLimiter = require("../middleware/loginLimiter");
```

Apply it to login:

```
router.post(
  "/login",
  loginLimiter,
  loginValidation,
  validateRequest,
  authController.login
);
```

For multiple production server instances, use a shared rate-limit store rather than relying on process-local memory.

Activity 14.7 — Optimize MongoDB queries

For read-only queries, use .lean() where you don't need Mongoose document methods:

```
const courses = await Course.find()
  .sort({ createdAt: -1 })
  .lean();
```

Add appropriate indexes:

```
userSchema.index({ email: 1 }, { unique: true });
```

If email already has unique: true, don't add a duplicate index declaration.

Enrollment and message indexes were included in Task 11.

For large collections, add pagination rather than returning every document.

Example:

```
const page = Math.max(
  1,
  Number.parseInt(req.query.page, 10) || 1
);
```

```
const limit = 20;


const courses = await Course.find()
  .skip((page - 1) * limit)
  .limit(limit)
  .lean();
```

Validate and cap pagination inputs to avoid excessive queries.

Activity 14.8 — Add logging

Install:

```
npm install morgan
```

UPDATE: backend/server.js

```
const morgan = require("morgan");


app.use(morgan("combined"));
```

Avoid logging passwords, JWT tokens, sensitive request bodies, or database credentials.

Activity 14.9 — Secure environment variables

UPDATE: backend/.gitignore

```
node_modules/
.env
.env.*
!.env.example
dist/
coverage/
```

NEW: backend/.env.example

```
PORT=5000
MONGO_URI=your_mongodb_connection_string
JWT_SECRET=your_random_secret
```

JWT_EXPIRES_IN=1h

CLIENT_URL=http://localhost:5173

Never put real credentials in .env.example.

If secrets were previously committed to GitHub, rotate them; adding .gitignore alone doesn't remove them from Git history.

Activity 14.10 — Handle common server errors

In your backend startup code, add process-level logging and controlled shutdown behavior. At minimum:

```
process.on("unhandledRejection", (error) => {
  console.error("Unhandled rejection:", error);
  process.exit(1);
});


process.on("uncaughtException", (error) => {
  console.error("Uncaught exception:", error);
  process.exit(1);
});
```

For production, use a process manager or hosting platform that restarts failed instances. Implement graceful shutdown for active HTTP connections and MongoDB rather than relying only on immediate process exit.

Activity 14.11 — Security testing

Test	Expected
Invalid registration email	400
Missing required fields	400
Duplicate email	409
Incorrect login password	401
Missing JWT	401
Expired JWT	401

Student accessing admin endpoint	403
Invalid MongoDB ID	400
Unknown API route	404
Unexpected server error	Controlled 500 response
Excessive login attempts	429
Password inspection in MongoDB	Hashed password

Also test that users cannot update another student's profile or access another user's messages.

Task 14 deliverable: A more secure, validated, logged, and error-handled application.

Task 15 — Deploy Application & Perform Testing

We'll deploy:

- React frontend: Vercel
- Express backend: Render
- MongoDB database: MongoDB Atlas

Activity 15.1 — Prepare the backend for production

UPDATE: backend/package.json

Make sure your scripts include the correct server entry point:

```
{
  "scripts": {
    "start": "node server.js"
  }
}
```

Keep your existing dependencies and other scripts. Do not replace the entire package.json with this snippet.

Verify locally:

```
npm start
```

Your backend must listen on:

const PORT = process.env.PORT || 5000;

Render supplies the production PORT environment variable.

Activity 15.2 — Prepare the frontend

UPDATE: frontend/.env.example

VITE_API_URL=http://localhost:5000/api/v1

VITE_SOCKET_URL=http://localhost:5000

UPDATE: frontend/src/api/apiClient.js

Ensure the Axios base URL comes from the environment:

```
const apiClient = axios.create({
  baseURL: import.meta.env.VITE_API_URL
});
```

UPDATE: frontend/src/api/socketClient.js

Ensure the Socket.IO client uses:

const SOCKET_URL = import.meta.env.VITE_SOCKET_URL;

Check the production build:

npm run build

For a Vite React project, this normally generates a dist folder.

Activity 15.3 — Set up MongoDB Atlas

1. Sign in to MongoDB Atlas.
2. Create a project and database cluster.
3. Create a database user with a strong password.
4. Configure network access for your hosting environment.
5. Copy the MongoDB connection string.
6. Set the connection string as MONGO_URI on Render.

Use the least database permissions required. Avoid exposing Atlas to unrestricted networks where a narrower access configuration is available.

Do not paste your database connection string into React code or commit it to GitHub.

Activity 15.4 — Push the project to GitHub

From the project root:

git status

git add .

git commit -m "Prepare Faceprep full stack project for deployment"

git push

Make sure .env, secrets, and node_modules are excluded.

If the repository hasn't been initialized or connected to GitHub, complete that setup first.

Activity 15.5 — Deploy Express on Render

In Render:

1. Create a new Web Service.
2. Connect your GitHub repository.
3. Select the backend root directory if using a monorepo.
4. Choose the Node.js runtime.
5. Set the build command to npm ci if a valid lockfile exists, otherwise npm install.
6. Set the start command to npm start.
7. Add the backend environment variables.
8. Deploy the service.

Configure these environment variables in Render:

Variable	Value
NODE_ENV	production
MONGO_URI	Atlas connection string
JWT_SECRET	Strong random secret
JWT_EXPIRES_IN	1h
CLIENT_URL	Deployed frontend URL

Render provides PORT; you generally shouldn't hardcode it.

After deployment, your backend URL will resemble:

https://your-backend.onrender.com

This is only an example, not an actual deployed URL.

Activity 15.6 — Deploy React on Vercel

In Vercel:

1. Import your GitHub repository.
2. Select the frontend root directory if necessary.
3. Choose the Vite framework preset.
4. Use npm run build.
5. Use dist as the output directory.
6. Configure frontend environment variables.
7. Deploy.

Example production environment variables:

VITE_API_URL=https://your-backend.onrender.com/api/v1

VITE_SOCKET_URL=https://your-backend.onrender.com

Replace these example domains with your real Render URL.

After changing Vite environment variables, redeploy the frontend so the new values are included in the build.

Activity 15.7 — Configure React Router fallback

If your application uses React Router with browser history, refreshing a nested route such as /dashboard must serve the React application rather than return a 404.

NEW: frontend/vercel.json

```
{
  "rewrites": [
    {
      "source": "/(.*)",
      "destination": "/index.html"
    }
  ]
}
```

Use this for a Vite single-page application. Adjust it if your frontend has other server routes or special hosting behavior.

Activity 15.8 — Configure production CORS

UPDATE: Render environment variable:

CLIENT_URL=https://your-frontend.vercel.app

Use the actual deployed Vercel URL.

Your Express CORS and Socket.IO CORS configurations must permit that frontend origin.

Restart or redeploy the backend after changing the environment variable.

Activity 15.9 — Verify the backend health endpoint

The PDF's Task 8 includes a backend health endpoint, which is useful during deployment.

If it doesn't exist yet, add this to backend/server.js before the error handlers:

```
app.get("/api/health", (req, res) => {
  res.status(200).json({
    status: "ok",
    message: "API is running"
  });
});
```

After deployment, open:

https://your-backend.onrender.com/api/health

Expected response:

```
{
  "status": "ok",
  "message": "API is running"
}
```

This confirms that Express is responding, but it does not independently verify MongoDB health.

Activity 15.10 — Test all production features

Feature	Test
Registration	Create a new student
Login	Log in and receive JWT
Authentication	Protected routes reject missing tokens
Student CRUD	Admin creates, views, updates, deletes test students
Course CRUD	Admin creates, updates, deletes a test course
Database	Data persists after backend restart
Enrollment	Student enrolls in a course
Learning progress	Student updates progress
Dashboard	Summary data loads
Chat	Two students exchange messages
Notifications	Recipient receives a notification without refreshing
Responsive design	Test desktop, tablet, mobile
API errors	Invalid input returns appropriate responses
Logout	Frontend clears the authenticated session

Test both the frontend and backend production URLs.

Activity 15.11 — Fix common deployment errors

Error	What to check
MongoDB connection failed	Atlas credentials and network access
CORS blocked	CLIENT_URL and allowed origins
API returns 404	Frontend API URL and route prefix
JWT verification failed	Backend JWT configuration and token validity
Socket.IO connection failed	Socket origin, URL, authentication, hosting support

React refresh returns 404	Vercel rewrite configuration
Blank React page	Build logs, browser console, frontend environment variables
Backend startup failed	Render logs, start command, Node version

Activity 15.12 — Update GitHub README

UPDATE: README.md

Add the following deployment section, replacing the placeholders after successful deployment.

Deployment

Live Application

- **Frontend:** https://your-frontend.vercel.app
- **Backend:** https://your-backend.onrender.com
- **API Health:** https://your-backend.onrender.com/api/health

Technology Stack

- React, React Router, Axios
- Node.js and Express
- MongoDB and Mongoose
- JWT and bcryptjs
- Socket.IO

Features

- Student registration and login
- JWT authentication
- Admin and student role-based access
- Student and Course CRUD
- Course enrollment and unenrollment
- Learning progress tracking
- Real-time chat and notifications
- Responsive frontend

Local Setup

1. Clone the GitHub repository.
2. Install backend dependencies using npm install.
3. Create the backend .env file using .env.example.
4. Configure MongoDB and JWT environment variables.
5. Start the backend using npm start.
6. Install frontend dependencies using npm install.
7. Create the frontend .env file with the API and Socket.IO URLs.
8. Start the frontend using npm run dev.

Deployment

The frontend is hosted on Vercel, the backend on Render, and the production database on MongoDB Atlas.

Testing

The project is tested using Postman and browser-based functional, integration, responsive, authentication, and real-time communication tests.

Only state that production tests have passed after actually completing them.

Task 15 deliverable: Live frontend, backend API, production MongoDB, functioning real-time features, completed testing, and updated GitHub documentation.

Final project structure

After completing Tasks 9–15, your project should have a structure similar to this:

student-course-management/

│

```
├── backend/
│   ├── config/
│   │   ├── db.js
│   │   └── socket.js
│   ├── controllers/
│   │   ├── authController.js
│   │   ├── studentController.js
│   │   ├── courseController.js
│   │   ├── profileController.js
│   │   ├── enrollmentController.js
│   │   └── learningProgressController.js
│   ├── middleware/
│   │   ├── authMiddleware.js
│   │   ├── roleMiddleware.js
│   │   ├── validateRequest.js
│   │   ├── loginLimiter.js
│   │   └── errorHandler.js
│   ├── models/
│   │   ├── User.js
│   │   ├── Course.js
│   │   ├── Enrollment.js
│   │   ├── LearningProgress.js
│   │   ├── Notification.js
│   │   └── Message.js
│   ├── routes/
│   │   ├── authRoutes.js
│   │   ├── studentRoutes.js
```

```
│   │   ├── courseRoutes.js
│   │   ├── profileRoutes.js
│   │   ├── enrollmentRoutes.js
│   │   ├── learningProgressRoutes.js
│   │   ├── messageRoutes.js
│   │   └── notificationRoutes.js
│   ├── validators/
│   │   └── authValidators.js
│   ├── scripts/
│   │   └── createAdmin.js
│   ├── .env
│   ├── .env.example
│   ├── .gitignore
│   ├── package.json
│   └── server.js
│
├── frontend/
│   ├── src/
│   │   ├── api/
│   │   │   ├── apiClient.js
│   │   │   └── socketClient.js
│   │   ├── components/
│   │   │   ├── Login.jsx
│   │   │   ├── Register.jsx
│   │   │   ├── CourseList.jsx
│   │   │   ├── MyCourses.jsx
│   │   │   ├── MyLearningProgress.jsx
```

```
│   │   │   ├── Chat.jsx
│   │   │   └── Notifications.jsx
│   │   └── App.jsx
│   ├── .env.example
│   ├── vercel.json
│   └── package.json
│
├── docs/
│   └── database-schema.md
│
└── README.md
```

This is a reference structure, not a requirement to rename all your existing files.

Important integration checks before calling the project finished

In particular, check these areas while integrating:

- Existing authentication: Keep one User model, one password-hashing strategy, and one login flow. Your admin creation script must use the same model and hashing strategy.
- Enrollment history: If you switch to withdrawn status, update both the unenrollment and re-enrollment controllers consistently.
- Student deletion: Add cleanup or anonymization of dependent records before treating deletion as production-ready.
- WebSocket security: Add rate limits, recipient authorization, and production connection handling before public deployment.
- Frontend integration: Merge components into your existing React Router structure and authentication state rather than duplicating pages.